

Software Requirements Specification

Restaurant App MVP (Frontend Only, React Native)

Assignment No. 1 — Fall 2026

Name: Muhammad Ahmed Raza

Registration No: 9605

Version 1.0

1. Introduction

1.1 Purpose

This document defines the software requirements for Version 1.0 of the Restaurant App MVP, a frontend-only React Native application. It describes the features, user roles, data, and behaviour that the app must provide so that a customer can browse a menu, search and order food, and reserve a table, and so that a restaurant manager can manage orders, reservations, and menu items from a phone. The intended audience is the course instructor, the development team (a single student developer for this assignment), and any reviewer assessing the app against its requirements.

1.2 Scope

In scope for Version 1.0:

- User signup and login for two roles: Customer and Manager, checked against local mock data (no real backend).
- Browsing the menu by category, viewing item details, and seeing Daily Special items.
- Searching and sorting menu items, with debounced search and recent search suggestions.
- Adding items to a cart, adjusting quantity, adding special instructions, and applying a mock promo code.
- Viewing an order summary with subtotal, service charge, tax, discount, and grand total.
- Reserving a table for a chosen date, time slot, and party size, with availability checked against mock tables.
- Placing an order as Dine-in or Takeaway and tracking its status through a demo timer.
- A Manager Dashboard for managing incoming orders, reservations, and menu items.
- Light and dark theme, shared across the whole app through Context.
- Local persistence of orders, reservations, and menu edits using AsyncStorage.

Out of scope for Version 1.0:

- Real payment processing of any kind (credit card, wallet, or cash handling logic).
- Push notifications or any server-sent alerts.
- A backend server, REST API, or database. All data is mock JSON/JavaScript held in React state and AsyncStorage.
- Real-time communication between multiple devices (for example, a manager's status change is not pushed live to another customer's phone; it is read from shared local state during this single-device demo).
- Multi-restaurant or multi-branch support; the app models one restaurant only.

1.3 Definitions and Acronyms

SRS: Software Requirements Specification, this document.

MVP: Minimum Viable Product, the smallest working version of the app with the core features.

UML: Unified Modeling Language, the diagram notation used to design the system before coding.

Hook: A React function (for example `useState` or `useEffect`) that lets a functional component use state or lifecycle features.

Context: React's Context API, used to share data such as the logged-in user or theme across many screens without passing props manually.

Reducer: A pure function used with `useReducer` that takes the current state and an action, and returns a new state.

Mock Data: Sample data written directly in JavaScript or JSON files inside the app, used in place of a real database.

FlatList: A React Native component that renders long scrollable lists efficiently by only drawing the items currently on screen.

AsyncStorage: A simple, local, key-value storage system for React Native, used here to save orders, reservations, and menu edits on the device.

Debounce: A technique that delays running a function until a short pause in activity (for example, typing) has passed.

2. Overall Description

2.1 User Roles

Role	Key Permissions & Goals
Customer	Can sign up and log in, browse the menu by category, search and sort items, add items to a cart, apply a promo code, reserve a table, place an order (dine-in or takeaway), track order status in real time, and view or cancel their own reservations. Goal: order food or book a table quickly without calling the restaurant.
Restaurant Manager	Can log in to a separate Manager Dashboard, view and update the status of incoming orders, accept or decline reservations, and add, edit, or mark menu items as unavailable. Goal: digitize day-to-day restaurant operations without a phone-in order system.

2.2 User Stories

- US-01 (Customer): As a customer, I want to browse the daily specials as soon as I open the app, so that I can decide what to order without reading the whole menu.
- US-02 (Customer): As a customer, I want to search for a dish by name, so that I can find it quickly instead of scrolling through every category.
- US-03 (Customer): As a customer, I want to book a table for a specific time such as 7:00 PM, so that my food is ready when I arrive.
- US-04 (Customer): As a customer, I want to add items to a cart and adjust their quantity, so that I can order more than one thing in a single checkout.
- US-05 (Customer): As a customer, I want to apply a promo code at checkout, so that I can get a discount on my order.

- US-06 (Customer): As a customer, I want to track my order status after placing it, so that I know when my food will be ready.
- US-07 (Manager): As a restaurant manager, I want to see incoming orders in one place, so that I no longer have to take orders over the phone.
- US-08 (Manager): As a restaurant manager, I want to accept or decline table reservations, so that I can manage seating capacity for the evening.
- US-09 (Manager): As a restaurant manager, I want to mark an item as unavailable when it runs out, so that customers cannot order something the kitchen cannot serve.
- US-10 (Manager): As a restaurant manager, I want to update an order's status as it moves through the kitchen, so that the customer sees accurate progress.

3. Functional Requirements

Each requirement below is testable and describes an observable system behaviour, grouped by module.

3.1 Authentication

FR-01: The system shall allow a new user to sign up with full name, email, password, confirm password, and a role selection of Customer or Manager.

FR-02: The system shall validate the signup form and show an inline error under a field the moment its value fails validation.

FR-03: The system shall authenticate a login attempt against the mock users array in `src/data/users.js` and show a one second loading state before responding.

FR-04: The system shall route a Customer to the Menu screen and a Manager to the Manager Dashboard immediately after a successful login.

3.2 Menu

FR-05: The system shall load the menu from a local mock data file and display a loading indicator while it loads.

FR-06: The system shall show an error message with a Retry button if the mock menu "fetch" fails or times out.

FR-07: The system shall display menu items in a scrollable list grouped by category using category chips.

FR-08: The system shall mark items with `isSpecial` true with a visible "Daily Special" badge.

FR-09: The system shall grey out any item with `isAvailable` false and disable its Add to Cart button.

3.3 Search

FR-10: The system shall let the customer search menu items by name using a text field, with the list updating 400 milliseconds after the user stops typing.

FR-11: The system shall let the customer sort the visible items by price (low to high, high to low) or by name (A to Z).

FR-12: The system shall show the five most recent search terms as tappable suggestions when the search field is empty and focused.

3.4 Cart

FR-13: The system shall allow a customer to add a menu item to the cart from the Menu screen.

FR-14: The system shall allow the customer to increase or decrease the quantity of a cart item, removing the item automatically when its quantity reaches zero.

FR-15: The system shall allow the customer to add a short special instruction (for example "no onions") to any cart item.

FR-16: The system shall allow the customer to enter a promo code, apply a valid discount, or show an error for an invalid code.

3.5 Reservation

FR-17: The system shall let the customer choose a date, an hourly time slot between 12:00 and 22:00, and a party size between 1 and 12.

FR-18: The system shall disable any time slot for which no table with enough seats is free.

FR-19: The system shall validate that the reservation is at least one hour in the future and that the phone number matches the Pakistani mobile format 03XX-XXXXXXX.

FR-20: The system shall show a confirmation summary before saving a reservation and allow the customer to cancel an existing reservation after an Alert confirmation.

3.6 Orders

FR-21: The system shall let the customer choose Dine-in with a table or Takeaway with a pickup time before placing an order.

FR-22: The system shall create a new order with status Pending as soon as the customer confirms checkout.

FR-23: The system shall automatically move an order from Pending to Preparing, then Ready, then Served, on a fixed demo timer, and display a step progress indicator.

FR-24: The system shall show a running elapsed time counter on the Order Tracking screen that updates every second.

3.7 Manager Dashboard

FR-25: The system shall show the Manager Dashboard tab only when the logged in user's role is manager.

FR-26: The system shall let the manager view all incoming orders and change an order's status manually from a fixed set of buttons.

FR-27: The system shall let the manager accept or decline a pending reservation.

FR-28: The system shall let the manager add a new menu item, edit an existing item's price, or toggle its availability, with the change visible on the customer Menu screen immediately.

4. Non-Functional Requirements

4.1 Usability

The app shall use large, clearly labelled touch targets, and every form field shall show its error message directly below the field, in plain language, as soon as the field becomes invalid. A first-time user shall be able to place a takeaway order without help within three minutes.

4.2 Performance

Screens shall render their main content within 2 seconds on a mid-range device once mock data has "loaded" (including the simulated 1.5 second delay). The menu list shall use FlatList so that scrolling stays smooth (no visible frame drops) even with fifty or more items, since FlatList only renders the rows currently visible on screen.

4.3 Responsiveness

Layouts shall use flexible units (percentages and flexbox) rather than fixed pixel widths, so that all screens remain usable on phone screens ranging from small (5-inch) to large (6.7-inch) without horizontal scrolling or cut-off buttons, and shall respect device safe areas.

4.4 Maintainability

Source code shall be organised under src into components, screens, context, reducers, hooks, data, and navigation folders. Repeated UI, such as a menu card, shall exist as a single reusable component rather than being copied across screens, so that a future change only needs to happen in one place.

4.5 Data Handling (No Backend)

Because there is no server, all data starts as mock JSON/JavaScript files bundled with the app. Any data created or changed during use, such as new orders, reservations, or menu edits, shall be written to AsyncStorage so it survives closing and reopening the app, and shall be reloaded before the first screen is shown so the app never briefly displays empty data.

5. Client-Side Data Model (Mock Data)

Since the app has no database, the data sets below are stored as local JSON/JavaScript files and held in React state during use.

Data Set Name	Fields	Description
Users	id, fullName, email, password, role (customer/manager)	Stored in src/data/users.js. Used by the Login and Signup screen to check credentials and to decide which screen to open after login.
Categories	id, name (Starters, Mains, Desserts, Drinks)	Stored in src/data/menu.js. Used to render the horizontal category chips on the Menu screen and to filter items.
MenuItems	id, name, description, price, category, image, isSpecial, isAvailable	Stored in src/data/menu.js, at least fifteen items. isSpecial shows a Daily Special badge. isAvailable greys out the card and disables the Add button.
Tables	id, seats, isAvailableAtSlot(time)	Stored in src/data/tables.js. Used by the Reservation screen to check which tables

		have enough seats free at a chosen time slot.
Reservations	id, customerId, date, timeSlot, partySize, tableId, contactPhone, status	Held in state through useReservation and persisted with AsyncStorage. Checked against Tables to disable full time slots.
Orders	id, customerId, items[], total, type (Dine-in/Takeaway), tableId or pickupTime, status, timestamp	Created by CartContext on checkout and managed by OrdersContext (useReducer). status moves through Pending, Preparing, Ready, Served.
Cart	items[](menuItemId, quantity, note), promoCode, discountPercent	Held only in memory through useReducer inside CartContext, and cleared once an order is placed.

6. UML Diagrams

The five diagrams below are produced separately in Question 2 and are placed here as placeholders with a caption and short explanation. The finished diagram images will be inserted in place of each placeholder box before submission.

Figure 1: Use Case Diagram

[Diagram image to be inserted here]

Shows the Customer and Manager actors and at least twelve use cases such as Sign Up, Log In, Browse Menu, Search Menu, Add to Cart, Apply Promo Code, Reserve Table, Place Order, Track Order, Manage Menu, Manage Reservations, and Update Order Status, including an include relationship from Place Order to Apply Promo Code.

Figure 2: Class Diagram

[Diagram image to be inserted here]

Shows the classes User, Customer, Manager, Category, MenuItem, Cart, CartItem, Order, Reservation, and Table, with Customer and Manager inheriting from User, their attributes and types, key operations, and the multiplicities between Cart and CartItem, and Order and MenuItem.

Figure 3: Sequence Diagram — Customer Places an Order

[Diagram image to be inserted here]

Shows the message flow from the customer tapping Add to Cart, through the Cart and Order Summary screens, to CartContext creating an order in OrdersContext, ending with the app opening the Order Tracking screen.

Figure 4: State Machine Diagram — Order Lifecycle

[Diagram image to be inserted here]

Shows the Order states Pending, Preparing, Ready, Served, and Cancelled, with the timer events and manager actions that move an order between them.

Figure 5: Component Diagram

[Diagram image to be inserted here]

Shows the React Native app's screens, the AuthContext, ThemeContext, CartContext, and OrdersContext providers, the cartReducer and ordersReducer, and the custom hooks (useForm, useDebounce, useReservation), with arrows showing which pieces depend on which.

7. MVP Frontend Development (React Native)

For each screen, this section states its purpose, its main UI elements, how a user enters and leaves it, the local data it uses, and the hooks it relies on.

7.1 Login and Signup Screen

Purpose: Lets a user log in with an existing account or sign up for a new one, and routes them to the correct home screen based on role.

UI elements: Mode toggle (Login/Signup), email field, password field with show/hide icon, full name and confirm password and role selector (shown only in Signup mode), inline error text, a submit button, and an ActivityIndicator while submitting.

Navigation: Entry: the first screen shown when no user is logged in. Exit: to the Menu screen (Customer) or Manager Dashboard (Manager) on success; stays on screen with an Alert on failure.

Local data used: src/data/users.js (mock users array).

Hooks used and why: useState for mode, form fields, errors object, show-password toggle, and isSubmitting flag; state is used because each keystroke and validation result must trigger a re-render of the form.

7.2 Menu Browsing Screen

Purpose: Shows the restaurant's menu, grouped by category, and lets the customer add items to the cart.

UI elements: Category chips, a FlatList of menu item cards (name, price, image, Daily Special badge), Add to Cart button per card, loading spinner, error view with Retry button, pull-to-refresh.

Navigation: Entry: after login (Customer) or from the bottom tab bar. Exit: to the Cart screen (via tab) or to a search/sort view on the same screen.

Local data used: src/data/menu.js (menu items and categories); read/write access to CartContext.

Hooks used and why: useState for menuItems, isLoading, error, selectedCategory, and filteredItems; useEffect (empty dependency array) to load the mock menu once and clean up its timer on unmount, and a second useEffect that depends on selectedCategory and menuItems to recompute filteredItems.

7.3 Search and Scroll Controls

Purpose: Adds fast, responsive search and easy navigation of a long menu list on top of the Menu screen.

UI elements: Search input with a focus icon and clear button, recent-search suggestion chips, a floating Back to top button, and a small debug label showing a render counter.

Navigation: Entry/exit: part of the Menu screen, so it shares the same entry and exit points as Section 7.2.

Local data used: Reads from the same src/data/menu.js list; keeps its own list of the last five search terms in state.

Hooks used and why: useRef for the TextInput (to focus it programmatically), for the debounce timeout id, for the FlatList (to call scrollToOffset), for the previous query (to block duplicate consecutive searches), and for the render counter, because none of these should cause a re-render by themselves the way state would.

7.4 Profile and Theme Screen

Purpose: Shows the logged-in user's details and lets them switch between light and dark mode.

UI elements: Avatar/initials, name, email, role, a light/dark mode switch, and a logout button.

Navigation: Entry: from the bottom tab bar (Customer) or a menu option (Manager). Exit: logout returns the user to the Login screen and clears the navigation stack.

Local data used: Reads the current user from AuthContext and the current theme from ThemeContext.

Hooks used and why: useContext through the custom useAuth and useTheme hooks, so the screen can read and update shared app-wide state without props being passed down from a parent screen.

7.5 Cart Screen

Purpose: Lets the customer review, adjust, and apply a promo code to the items they intend to order.

UI elements: A list of cart items with quantity steppers, a remove button, a note field per item, a promo code input, and a Proceed to Checkout button.

Navigation: Entry: from the cart tab icon (which also shows a live item count badge). Exit: to the Order Summary screen.

Local data used: Cart state (items, promoCode, discountPercent) and a small local object of valid promo codes.

Hooks used and why: useReducer through CartContext/useCart, because adding, removing, incrementing, and applying a promo code are related state changes best expressed as a single pure reducer rather than several separate useState calls.

7.6 Order Summary Screen

Purpose: Shows the final cost breakdown before the customer confirms the order.

UI elements: A read-only breakdown of subtotal, service charge, tax, discount, and grand total, plus a Dine-in/Takeaway choice and a Confirm Order button.

Navigation: Entry: from the Cart screen's Proceed to Checkout button. Exit: to the Order Tracking screen once the order is confirmed.

Local data used: Reads cart items and discount from CartContext; writes a new order through OrdersContext.

Hooks used and why: useMemo to recompute the totals only when cart items or the discount change; useCallback to keep the Confirm handler's reference stable if it is passed into a memoised child component.

7.7 Table Reservation Screen

Purpose: Lets the customer book a table for a chosen date, time, and party size.

UI elements: Date picker, hourly time-slot buttons (disabled when full), a party-size stepper, a phone number field, a confirmation modal, and a My Reservations list with cancel buttons.

Navigation: Entry: from the bottom tab bar. Exit: stays on screen after a successful booking, showing the new entry in My Reservations; a cancel returns to the same list.

Local data used: src/data/tables.js (mock tables) and the reservations list held by useReservation.

Hooks used and why: A custom hook, useReservation, holds all the business logic (availability check, validation, create/cancel), so the screen itself only renders UI and calls the hook's returned functions.

7.8 Order Tracking and Manager Dashboard

Purpose: Lets the customer watch their order's status change in real time, and lets the manager run the restaurant side of the app.

UI elements: Order Tracking: step progress bar (Pending, Preparing, Ready, Served) and an elapsed-time counter. Manager Dashboard: three tabs — Incoming Orders (status buttons), Reservations (accept/decline), and Menu Management (add/edit/toggle items).

Navigation: Entry: Order Tracking opens right after Order Summary confirms; the Manager Dashboard tab is only visible in the bottom tabs when the logged-in user's role is manager. Exit: back to the Menu screen (Customer) or staying within the Dashboard tabs (Manager).

Local data used: The shared Orders, Reservations, and MenuItem data, all persisted to AsyncStorage.

Hooks used and why: useEffect with setInterval to auto-advance a demo order's status and to tick the elapsed-time counter every second, clearing both intervals on unmount; useEffect to load and save AsyncStorage data on app start and whenever it changes; this screen also draws on useState, useContext, and useReducer already introduced above, bringing every hook together in one place.